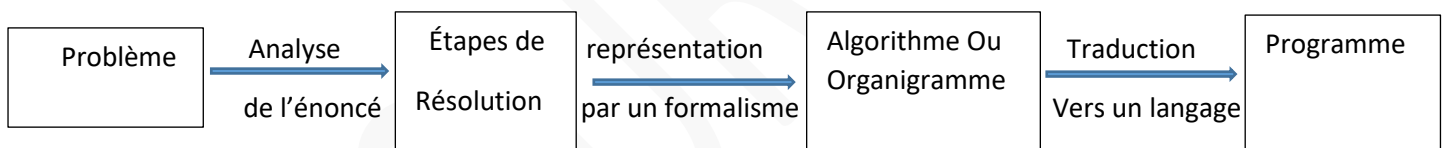


Introduction

Le langage C a été créé au début des années 70, dans le but de permettre le développement du système d'exploitation UNIX portable, avant qu'il soit utilisé comme langage de programmation de haut niveau.

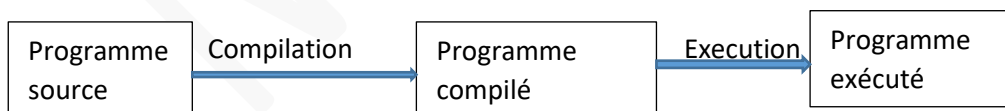
Le langage C, est un des langages les plus utilisés actuellement, dû au fait qu'il comporte des instructions et des structures de haut niveau, il est aussi doté d'un compilateur très performant. Le langage est caractérisé par sa portabilité, un programme écrit en C est portable sans modification sur n'importe quel système d'exploitation doté d'un compilateur C, les programmes écrits en C sont rapides.

La résolution informatique d'un problème passe par plusieurs étapes qu'on peut schématiser comme suit :



Une fois le programme écrit il va falloir le vérifier et le corriger en utilisant la compilation, le compilateur décrit les erreurs de syntaxe du programme mais pas les erreurs de logiques.

Ensuite le programme sans erreurs est exécuté, donc on exploite le programme et on vérifie les résultats.



Chapitre I : Les éléments du langage C

I. Structure d'un programme en C :

La structure générale d'un programme en C est la suivante

Structure d'un algorithme	Structure d'un programme en C
Algorithme <identificateur> <Déclarations> Debut <Instructions/Opérations> Fin.	Bibliothèque & directives main() { <Déclarations> <Instructions> }

Un programme en C doit respecter les règles de base suivantes :

- La fonction main() est la fonction principale des programmes en C. Elle se trouve obligatoirement dans tous les programmes.
- Chaque instruction se termine par un ;
- Le C distingue entre les majuscules et les minuscules.

1-1 Les bibliothèques

C'est des fichiers qui contiennent des fonctions (entrée/sortie, mathématique, traitement de chaîne de caractères, etc..) prédéfinies et utilisées dans le langage C. Pour utiliser les fonctions d'une bibliothèque on introduit le fichier entête de cette bibliothèque dans la partie entête du programme avec la syntaxe suivante : #include<nom de la bibliothèque.h>

Exemple : pour utiliser une fonction d'entrée sortie dans le programme, il faut inclure la bibliothèque stdio.lib à travers son fichier entete, stdio.h, en ajoutant l'instruction suivante dans le programme :

```
#include<stdio.h>
```

1-2 Les déclarations :

a) Déclaration des constantes

On déclare les constantes à l'aide de la directive **define** ou le mot clé **const**

En Algorithmique	En langage C deux possibilités	
Const <identificateur>=valeur ;	Avant main() #define <idf> valeur ;	Après main() const <type> <idf>=valeur ;
Exemple Const n=20	#define n 20 ;	const int n=20 ;

b) Déclaration de variables

En Algorithmique	En langage C
var <idf> : <type> ; <idf ₁ >, <idf ₂ >, ..., <idf _n > : <type> ;	<type> <idf> ; <type> <idf ₁ >, <idf ₂ >, ..., <idf _n > ;

b.1) les types de base dans C :

Type	signification
int	Entier
short int	Entier court
long int	Entier long
char	Caractère
float	Réel
double	Réel en double précision

Remarque1 : on peut initialiser une variable lors de sa déclaration

Exemple de déclarations

En Algorithmique	En langage C
Var l, j : entier ;	main()

a,b :reel; Debut j←0.....	{ int l,j=0; float a,b;
---	---------------------------------------

Remarque 2: le type booléen n'existe pas en langage C, on le remplace par un entier qui prend deux valeurs 0 et 1 pour désigner le vrai ou faux respectivement ou inversement ou par un caractère qui prend 'f' ou 'v' .

b.2) Les identificateurs :

Utilisés pour identifier une variable, constante, fonction, etc....c'est une suite de caractères parmi, les lettres en majuscules, minuscules mais pas accentuées, les chiffres le blanc souligné (tiret du 8_), le premier caractère ne peut pas être un chiffre.

2. Les commentaires

On peut ajouter les commentaires dans un programme de deux manières :

- les deux symboles // si le commentaire est sur une ligne.
- les symboles /*.....*/ , ouvrir le commentaire par /* et le fermer par */ utile pour les commentaire sur plusieurs lignes , mais aussi valable pour les commentaires sur une seule ligne.

3. Les fonctions d'Entrée/ Sortie :

3.1 La fonction d'écriture ou de sortie printf : c'est une fonction de la bibliothèque standard input output : **stdio.lib** . Elle permet d'afficher un message ou le contenu d'une ou plusieurs variables ou d'une expression (qui est évaluée en premier puis sa valeur est affichée).

a) Affichage d'un message :

En Algorithmique	En langage C
Ecrire('message');	printf("message");

b) Affichage d'une ou plusieurs valeurs :

En Algorithmique	En langage C
Une seule variable Ecrire(idf_var) ;	Une seule variable printf(" format ",idf_variable) ;
Plusieurs variables Ecrire(idf_var1,idf_var2, ...,idf_var _n);	Plusieurs variables printf("format ₁ format ₂ ...format _n ", idf_var1,idf_var2,...idf_var _n);
Un message et la valeur d'une variable Ecrire ('message',idf_var) ;	Un message et la valeur d'une variable printf("message format ",idf_var) ;
Plusieurs Ecrire('message ₁ ',idf_var1,'message ₂ ',idf_var2) ;	Plusieurs Printf("messag ₁ format ₁ message ₂ format ₂ ",idf_var1, idf_var2) ;

c) Affichage d'une expression:

En Algorithmique	En langage C
Ecrire('message',<exp>) ;	Printf("message format",<exp>);

Le format d'une variable dépend de son type

Types	Formats
Int /	%d
short int	%i
long int	%ld
char	%c
Chaine de caractères	%s
float	%f
double	%g

3.1 La fonction de lecture ou d'entrée scanf : c'est une fonction de la bibliothèque standard input output : stdio.lib . Elle permet la saisie des données au clavier . sa syntaxe est la suivante

a) Une seule variable

En Algorithmique	En langage C
lire(idf_var) ;	scanf("format", &idf_var);

b) plusieurs variables

En Algorithmique	En langage C
lire(idf_var ₁ , idf_var ₂ ,...,idf_var _n) ;	scanf("format ₁ format ₂ ...format _n ", &idf_var ₁ , &idf_var ₂ ,...,i&idf_var _n);

Exercice 1 : écrire un programme en C qui permet de définir la constante A=100 de deux manières différentes, puis l'afficher.

Avec #define

```
#include<stdio.h>
```

```
#define A 100
```

```
main()
```

```
{
```

```
printf("A=%d",A) ;
```

```
}
```

Avec const

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
const int A=100 ;
```

```
printf("A=%d",A) ;
```

```
}
```

Exercice 2 : écrire un programme en C qui permet de déclarer une variable de type caractère, de l'initialiser à la valeur 'd', puis l'afficher.

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
char Ca='d';
```

```
printf("Ca=%c",Ca);
```

```
}
```

Exercice 3 : écrire un programme en C qui permet d'afficher 'bonjour '

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
printf("bonjour");
```

```
}
```

Exercice 4 : écrire un programme qui permet de déclarer deux variables entières, une variable réel et un caractère, d'introduire leurs valeurs par clavier et de les afficher par la suite.

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int a,b ;
```

```
float c;
```

```
char j ;
```

```
printf("introduire deux entiers, un reel et un caractere respectivement\n");
```

```
scanf("%d %d %f %c",&a,&b,&c,&j);

printf("a=%d ,b=%d, c=%f, j=%c",a,b,c,j);

}
```

Exercice 5 écrire un programme qui permet d'afficher la valeur de l'expression $a+b-c*d$

Avec $a=20$, $b=30$, $c=100$, $d=4$.

```
#include<stdio.h>

main()

{

const int a=20,b=30,c=100,d=4 ;

printf("le resultat est %d ",a+b-c*d);

}
```

Exercice6 écrire un algorithme puis un programme en c qui permet de faire la somme de deux entiers :

```
#include<stdio.h>

main()

{ int A,B,S;

printf("Donnez les deux entiers");

scanf("%d %d",&A,&B);

S=A+B;

printf("la somme de %d+%d=%d",A,B,S);

}
```

3 Les opérateurs en langage C :

3.1 Les opérateurs arithmétiques :

Les opérateurs arithmétiques sont : l'opérateur unaire - qui permet de changer le signe d'un nombre, et les opérateurs binaires donnés dans le tableau suivant :

+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Modulo (reste de la division)

3.2 Les opérateurs de comparaison :

Les opérateurs de comparaison permettent de former des expressions de contrôle en comparant deux opérandes, ils sont présentés dans le tableau ci-dessous :

<	Infériorité stricte
<=	Infériorité large
>	Supériorité stricte
>=	Supériorité large
==	Egalité
!=	Inégalité

3.3 Les opérateurs logiques :

Ils permettent de réaliser des expressions logiques complexes à partir d'expressions simples

!	Négation logique
&&	ET logique
	OU logique

3.4 Les opérateurs d'incrément et de décrémentation :

++	Incrément	Ajouter une unité à une valeur
--	Décrément	Diminuer d'une unité une valeur

Exemple : $i=i+1$ et $i++$ sont équivalents

$i=i-1$ et $i--$ sont équivalents

4. Affectation :

4.1 Affectation simple :

Symbolisée par le signe (=). Sa syntaxe est : **Nom_Var=<expression>**

4.2 Affectation composée :

C'est une affectation avec calcul arithmétique.

+=	i+=20	i=i+20
-=	i-=k	i=i-k
=	i=k	i=i*k
/=	i/=k	i=i/k
%=	i%=k	i=i%k

5. Les instructions conditionnelles

5.1 L'instruction conditionnelle tronquée :

Algorithmique	Langage C
si <condition > alors <bloc d'instructions> Fsi ;	if (condition) instruction ; if (condition) { instruction 1 ; instruction2 ; Instruction n ; }

Exemple : écrire un programme qui demande d'introduire un entier positif, de vérifier s'il l'est et affiche "entier positif."

```
#include<stdio.h>
```

```
main()
```

```
{
```

```

int C;

printf("donner un nombre\n");

scanf("%d",&C);

if(C>0) printf("entier positif");

}

```

5.2 L'instruction conditionnelle alternative

Algorithmique	Langage C
si <condition > alors <bloc d'instructions1> sinon <bloc d'instructions2> fsi ;	if (condition) instruction1 ; else instruction2 if (condition) { instruction1 1 ; instruction12 ; Instruction 1n ; } else {instruction2 1 ; instruction2 2 ; Instruction 2 n ; }

Exemple : écrire un programme qui demande d'introduire un entier positif, de vérifier s'il l'est et affiche "entier positif", sinon afficher "le nombre n'est pas positif".

```

#include<stdio.h>

main()

{

int C;

printf("donner un nombre");

scanf("%d",&C);

```

```
if(C>0) printf("entier positif");  
  
else printf("entier non positif");  
  
}
```

Exemple2 : écrire un programme qui permet d'afficher si un nombre entier introduit est positif, négatif ou nul.

```
#include<stdio.h>  
  
main()  
{  
  
int C;  
  
printf("donner un nombre\n");  
  
scanf("%d",&C);  
  
if(C>0) printf("entier positif");  
  
else {  
  
    if(C<0) printf("entier negatif");  
  
    else printf("entier nul");  
  
}  
  
}
```

Exemple3 : écrire un programme qui permet de déterminer le maximum entre trois entiers A,B,C.

```
#include<stdio.h>  
  
main()  
{  
  
int A ,B,C,max;  
  
printf("Donner les entiers a comparer ");
```

```
scanf("%d %d %d", &A, &B, &C);

if(A>B && A>C){ max=A;}

        else{

                if(B>A && B>C ) { max=B;}

                        else {max=C;}

        }

printf("max=%d",max);

}
```

2eme solution:

```
#include<stdio.h>

main()

{

int A ,B,C,max;

printf("Donner les entiers a comparer ");

scanf("%d %d %d", &A, &B, &C);

if(A>B ) max=A;

else max=B;

if(max<C ) max=C;

printf("max=%d",max);

}
```

5.3 L'instruction conditionnelle aux choix multiples :

L'instruction **switch** permet des choix multiples uniquement sur des entiers ou des caractères.

Sa syntaxe est la suivante :

Algorithmique	Langage C
Cas <expression> Vaut Domaine1 : <bloc d'actions 1> ; Domaine 2 : <bloc d'actions 2> ; . . Domaine n : <bloc d'actions n> ; [Sinon <bloc d'actions p> ;] Fincas ;	switch (variable) { case valeur_1: <bloc d'instructions_1>; break; case valeur_2: <bloc d'instructions_2>; break; case valeur_n: <bloc d'instruction_n> ; break ; [default: <bloc d'instructions_d>;] } Variable est de type caractère ou entier.

Exemple écrire un programme en C qui affiche les noms des wilayas suivant leur code. On considère uniquement les numéros : 16, 31, 35, 42.

```
#include<stdio.h>
```

```
main()
```

```
{     int code;
```

```
    printf("donner un code wilaya parmi ces codes: 16,31,35,42\n");
```

```
    scanf("%d", &code);
```

```
    switch(code)
```

```
    {
```

```
        case 16: printf("16 correspond à wilaya d'Alger");
```

```
                break;
```

```
        case 31:printf("31 correspond à wilaya d'Oran");
```

```
                break;
```

```
        case 35:printf("35 correspond à wilaya de Boumerdès");
```

```

        break;

    case 42:printf("42 correspond à wilaya de Tipaza");

        break;

    default: printf("code non existant dans la liste proposée");

}

}

```

Exercice 2 : écrire un programme en C qui à partir d'un numéro de mois vous affiche le nombre de jours de ce mois.

Algorithme Jour_Mois ;
Var
M :entier ;
Debut
Ecrire('Introduire le numéro du mois') ;
Lire(M) ;
Cas M vaut
1,3,5,7,8,10,12 : **Ecrire**('le nombre de jours est 31') ;
4,6,9,11 : **Ecrire**('le nombre de jours est 30') ;
2 :**Ecrire**('le nombre de jours est 29 si l'année est bissextile, 28 sinon') ;
Sinon Ecrire('numéro incorrect') ;
Fincas ;
Fin.

```
#include<stdio.h>
```

```
main()
```

```

{    int M;

    printf("introduire le numéro du mois entre 1 et 12\n");

    scanf("%d", &M);

    switch(M)

    {

        case 1: case 3: case 5: case 7: case 8: case 10: case 12:

            printf("le nombre de jours est 31);

            break;

        case 4: case 6: case 9: case 11:printf("le nombre de jours est 30");

```

```

        break;

    case 2:printf("le nombre de jours est 29 si l'année est bissextile, 28 sinon");

        break;

    default: printf("numéro incorrect ");

}

}

```

Exercice3 Ecrire un programme pour la résolution d'une équation deuxième degré $ax^2+bx+c=0$

```

#include<stdio.h>

#include<math.h>

main()
{
    float a,b,c,x1,x2,d ;
    printf("Donner les trois coefficients :\n");
    scanf("%f %f %f",&a,&b,&c);
    printf("l'equation est %.1fx^2+%.1fx+%.1f=0 \n",a,b,c);
    if (a==0)
        if (b==0)
            if (c==0) printf("la solution est R");
            else printf("erreur il n'y a pas de solution");
        else { x1=(-c/b);
            printf("la solution est %.2f",x1);
        }
    else { d=pow(b,2)-4*a*c ;

```

```
if (d<0) printf ( "delta négatif: il n'y a pas de solution dans R");
    else if (d==0) { x1=-b/(2*a);
        printf("delta nul : la solution est double x1=x2=%.2f",x1);
    }
    else { x1=(-b-sqrt(d))/(2*a) ; x2=(-b+sqrt(d))/(2*a);
        printf(" delta positif : les solutions sont x1= %.2f , x2= %.2f",x1,x2);
    }
}
}
```

TP5 :

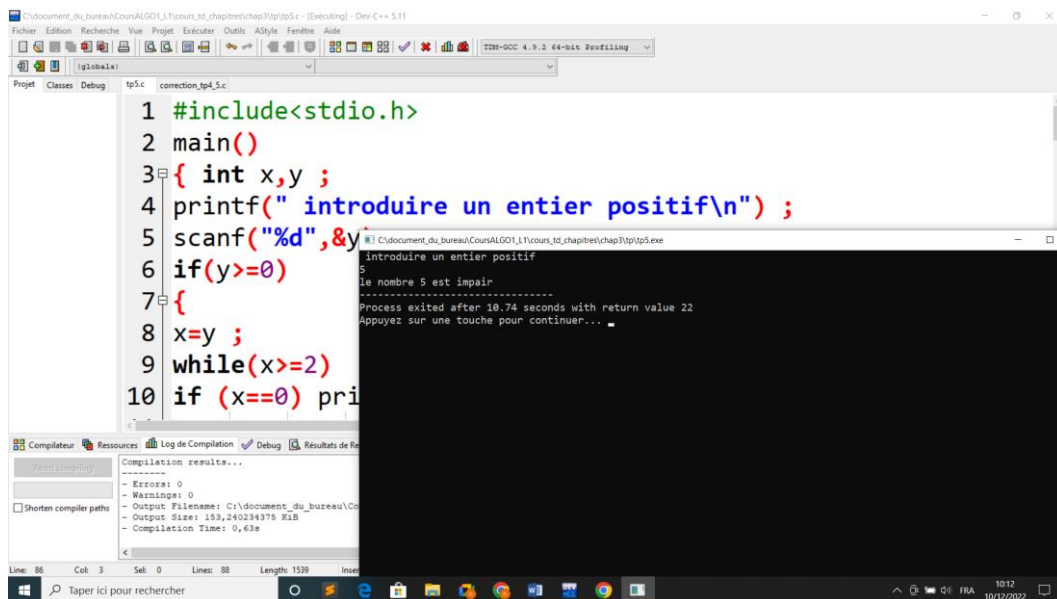
6. Les Instructions Itératives :

6.1) La boucle while (Tant que) :

Algorithmique	Langage C
<Initialisation> ; Tant que <condition> Faire <bloc d'instructions> Fait ;	<Initialisation> ; while(condition) instruction; <Initialisation> ; while(condition) { instruction 1; instruction2; ... instruction n; }

Exemple : écrire un algorithme puis un programme qui permet de déterminer si un nombre positif est pair sans utiliser l'opérateur de reste de la division.

Algorithme	Programme
Algorithme pair ; Var X, Y : entier ; debut ecrire ("donner un nombre positif") ; Lire(Y) ; Si y>=0 alors X ← Y ; Tant que (X>=2) Faire X ← X-2 ; Fait ; Si (X=0) alors écrire ('le nombre ', Y,'est pair ') Sinon écrire ('le nombre ', Y,'est impair ') Fsi Sinon écrire('le nombre introduit n''est pas positif ') Fsi ; Fin.	<pre>#include<stdio.h> main() { int x,y ; printf(" introduire un entier positif\n") ; scanf("%d",&y) ; if(y>=0) { x=y ; while(x>=2) x=x-2 ; if (x==0) printf("le nombre %d est pair",y) ; else printf("le nombre %d est impair",y) ; } else printf("le nombre introduit n'est pas positif ") ; }</pre>



Exercices : 1) écrire un programme qui permet d'afficher les diviseurs d'un nombre positif donné.

Solution

Algorithme Diviseur ;

Var

a,b,i,m :entier ;

debut

ecrire('donner un entier positif') ;

lire(a) ;

si a>=0 alors

si(a=0) alors écrire('tous les nombres sont diviseurs de 0 sauf 0)

sinon

$i \leftarrow 1$; $m \leftarrow a \text{ DIV } 2$;

Tantque $i \leq m$ faire

Si $a \text{ MOD } i = 0$ alors écrire(i,'est diviseur de',a) ;

fsi ;

$i \leftarrow i+1$;

fait ;

ecrire(a,'est diviseur de',a) ;

fsi

sinon écrire("le nombre est négatif") ;

fsi ;

fin.

Le programme :

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int a,i,m ;
```

```
printf("donnez un entier positif: ") ;
```

```
scanf("%d",&a);
```

```
if (a>=0)
```

```
{
```

```
if(a==0) printf("tous les nombres sont diviseurs de 0 sauf 0");
```

```
else
```

```
{
```

```
i=1; m← a/2;
```

```
printf("les diviseurs du nombre %d sont ",a);
```

```
while(i<=m)
```

```
{
```

```
if (a%i==0) printf(" %d ",i);
```

```
i++;
```

```
}
```

```
printf(" %d ",a);
```

```
}
```

```
}
```

```
else printf(" le nombre est negatif") ;
```

```
}
```

```

16 main()
17 {
18     int a,i,m ;
19     printf("donnez un entier positif: " );
20     scanf("%d",&a);
21     if (a>=0)
22     {
23         if(a==0) printf("tous les nombres sont diviseurs de 0 sauf 0")
24         else
25         {
26             i=1; m=a/2;
27             printf("les diviseurs du nombre %d sont ",a);
28             while(i<=m)
29             {
30                 if (a%i==0) printf(" %d ",i);
31                 i++;
32             }
33             printf(" %d ",a);
34         }
35     }
36     else printf(" le nombre est negatif" );

```

Compilation results...

```

- Errors: 0
- Warnings: 0
- Output Filename: C:\document_du_bureau\CoursALGO1_1\cours_td_chapitres\chap3\tp5.exe
- Output Size: 153,0712890625 KiB
- Compilation Time: 0,31s

```

Output window content:

```

C:\document_du_bureau\CoursALGO1_1\cours_td_chapitres\chap3\tp5.exe
donnez un entier positif: 24
les diviseurs du nombre 24 sont  1 2 3 4 6 8 12 24
process exited after 7.521 seconds with return value 4
appuyez sur une touche pour continuer...

```

A préparer

2) écrire un programme qui permet de faire la division entière (quotient et reste) entre deux entiers positifs.

6.2) La boucle do....while (repetier....jusqu'a):

do....while n'est pas vraiment équivalente à la boucle **repetier.. jusqu'à**, en effet la boucle **repetier** répète les instructions de la boucle jusqu'à ce que la condition soit vérifiée alors que la boucle **do..while** répète les instructions de la boucle tant que la négation de la condition est vérifiée.

Algorithmique	Langage C
repetier Action 1 ; Action 2 ; . . Action n ; <Agir sur condition > Jusqu'a (condition) ;	do instruction1 ; while (condition) ; do { inst1 ; inst2 ; inst n ; } while (condition) ;

Exemple : écrire un programme qui permet d'introduire un entier positif ou nul.

Solution :

Algorithme	Programme
Algorithme inst_repetier ; Var N :entier ; debut Repetier lire(N) ; jusqu'a N>=0 ; Fin.	<pre>#include <stdio.h> main() { int N ; do { printf(" donner un entier positif ou nul\n") ; scanf("%d",&N) ; } while (N<0) ;</pre>

The screenshot shows a C++ IDE with the following code in the editor:

```

34 }
35 }
36 else printf(" Le nombre est negatif") ;
37 }*/
38
39 #include <stdio.h>
40 main()
41 { int N ;
42 do {
43 printf(" donner un entier positif ou nul\n") ;
44 scanf("%d",&N) ;
45 } while (N<0) ;
46 }
47
48 #include<stdio.h>
49 main()
50 {
51 int nb ;
52 do {
53 printf("entrez un nombre entre 1 e
54 scanf("%d",&nb) ;

```

The output window shows the execution results:

```

donner un entier positif ou nul
-9
donner un entier positif ou nul
-7
donner un entier positif ou nul
2
-----
Process exited after 18.73 seconds with return value 2
Appuyez sur une touche pour continuer...

```

The compilation results window shows:

```

Compilation results...
-----
Errors: 0
Warnings: 0
Output Filename: C:\document_du_bureau\CoursALGO1_1\cours_td_chapitres\chap3\tp5.exe
Output Size: 152,5693359375 KiB
Compilation Time: 0,28s

```

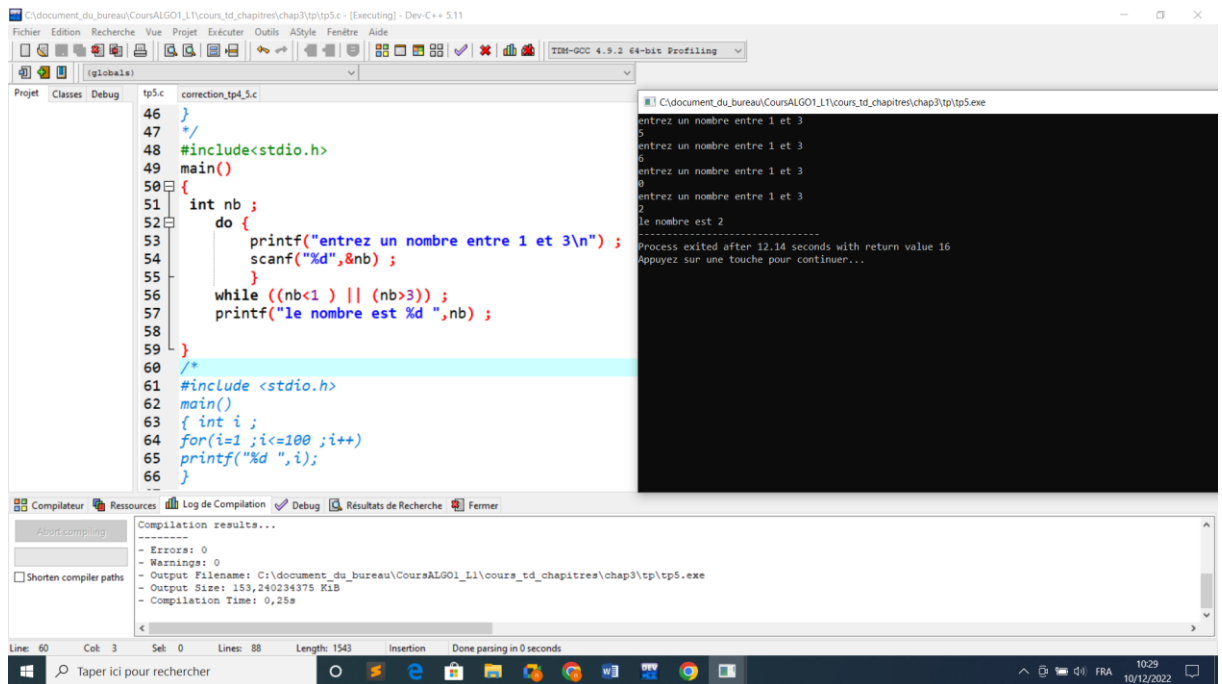
Exercice

- 1) écrire un algorithme et un prgm. Qui demande à l'utilisateur d'introduire un nombre entre 1 et 3 jusqu'à ce que la réponse soit juste.

Algorithme Nombre_1_3 ;
Var
Nb :entier ;
Debut
Repeter
Ecrire('donner un nombre entre 1 et 3') ;
Lire(nb) ;
Jusqu'à nb >= 1 et nb <= 3 ;
Ecrire(nb) ;
Fin.

Le programme :

```
#include<stdio.h>  
main()  
{  
int nb ;  
do {  
    printf("entrez un nombre entre 1 et 3") ;  
    scanf("%d",&nb) ;  
    }  
while ((nb<1 ) || (nb>3)) ;  
printf("le nombre est %d ",nb) ;  
}
```



A préparer

- 2) Soit un ensemble de caractères terminé par '.'. Ecrire un algorithme, puis un programme qui calcule le nombre de caractère de cet ensemble.

6.3) La boucle for (pour) :

Algorithmique	Langage C
Pour <ident> ← <expression1> a <expression2> faire <bloc d'instructions> Fait ; expression1 : valeur initiale du compteur expression 2 : valeur finale du compteur	<pre> for(exp1 ;exp2 ;exp3) inst1; for(exp1 ;exp2 ;exp3) { inst1; inst2; inst n; } Avec </pre>

	exp1: instruction d'initialisation du compteur exp2 : la condition d'arrêt de la boucle exp3 : incrémentation du compteur
--	--

Exemple : écrire un programme qui permet d'afficher les nombres naturels de 1 à 100.

Solution

Algorithme	Programme
Algorithme inst_pour ; Var i :entier ; debut pour $i \leftarrow 1$ a 100 faire Ecrire (i) ; fait ; Fin.	<pre>#include <stdio.h> main() { int i ; for(i=1 ;i<=100 ;i++) printf("%d ",i); }</pre>

```

52 do {
53     printf("entrez un nombre entre 1 et 100 : ");
54     scanf("%d", &nb);
55 } while ((nb < 1) || (nb > 100));
56 printf("Le nombre est %d", nb);
57
58
59
60
61 #include <stdio.h>
62 main()
63 { int i ;
64   for(i=1 ;i<=100 ;i++)
65     printf("%d ",i);
66 }
67
68 #include <stdio.h>
69 main()
70 {
71     int P,X,N,i ;
72     do

```

Process exited after 0.09817 seconds with return value 4
Appuyez sur une touche pour continuer...

Exercice

- 1) écrire l'algorithme et le programme qui permettent de faire le produit de N nombres entiers .

Algorithme produit ;

Var P, I, X, N : entier;

Debut

Repeter

Lire(N) ;

Jusqu'à N>0 ;

P $\leftarrow$ 1 ;

Pour I $\leftarrow$ 1 à N Faire

Lire (X) ;

P $\leftarrow$ P*X ;

Fait ;

Ecrire (P) ;

Fin.

Le programme :

#include<stdio.h>

main()

{

int P,X,N,i ;

do

{

printf("donner le nombre de nombre ");

scanf("%d",&N);

}while(N<=0);

P=1 ;

for (i=1 ; i<=N ; i++)

{

printf("donnez le nombre ");

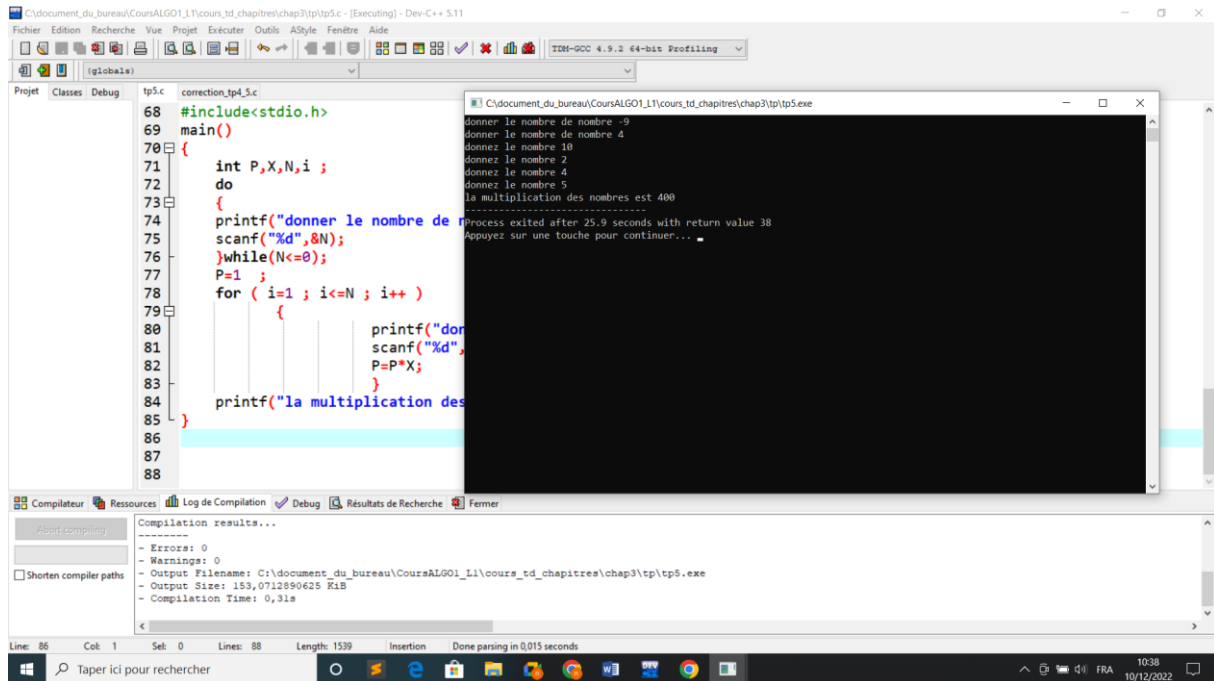
scanf("%d",&X);

```

        P=P*X;
    }

    printf("la multiplication des nombres est %d ",P);
}

```



A préparer

2) Écrire l'algorithme ensuite le programme qui calcule la factorielle d'un entier naturel

Exercice : Ecrire un programme qui permet de calculer la somme d'ordre N de S_n définie comme suit en utilisant seulement les opérateurs de base (sans l'utilisation de l'opérateur de puissance).

$$S_n = \sum_{i=0}^N \frac{(-1)^{i+1}}{x^i}$$

Solutions des exos à préparer :

1) **boucle while** : écrire un programme qui permet de faire la division entière (quotient et reste) entre deux entiers positifs.

Algorithme : vous le trouverez dans la solution de la série 1

Programme :

```

#include<stdio.h>

main()
{ int A,B,q,r ;

```

```

do{printf("donner deux entiers positifs\n");
    scanf("%d %d",&A,&B);
}
while (A<0 || B<=0);
q=0 ; r=A ;
while(r>=B)
{
    r=r-B ;
    q=q+1 ;
}
printf("le quotient de %d div %d=%d le reste = %d",A,B,q,r) ;
}

```

2)boucle do..while : soit un ensemble de caractères terminé par '.'. Ecrire un algorithme, puis un programme qui calcule le nombre de caractère de cet ensemble.

Solution :

```

Algorithme nb_caractere ;
Var
Cpt :entier ;
A :caractere ;
Debut
Repete ecrire('donner un caractere et . pour arreter') ;
    Lire(A) ;
    Cpt←cpt+1 ;
Jusqu'à A='.' ;
Ecrire('le nombre de caractères =',cpt-1) ;
Fin.

```

Programme

```
#include<stdio.h>
```

```
main()
```

```

{

int cpt ;

char A ;

cpt=0 ;

do {

    printf("entrez le caractere") ;

    scanf(" %c",&A) ;

    cpt=cpt+1 ;

}

while(A!='.') ;

printf("le nombre de caractere est %d",cpt-1) ; //cpt-1 pour ne pas compter le '.'

}

```

3) boucle for : écrire l'algorithme ensuite le programme qui calcule la factorielle d'un entier naturel N.

```

0 !=1
1! = 1
2! = 1 × 2 = 2
3! = 1 × 2 × 3 = 6
4! = 1 × 2 × 3 × 4 = 24
10! = 1 × 2 × 3 × 4 × 5 × 6 × 7 × 8 × 9 × 10 = 3 628 800

```

Algorithme

Algorithme Factorielle ;

Var

F,i,N :entier ;

Debut

repete

Ecrire('Donner un entier positif ');

Lire(N);

Jusqu'à $N \geq 0$;

$F \leftarrow 1$;

Pour $i \leftarrow 2$ à N faire

$F \leftarrow F * i$;

Fait;

Ecrire(N, '!=', F);

Fin.

Programme

```
#include<stdio.h>
```

```
main()
```

```
{ int N,i;
```

```
long long int F; /* long long int pour le résultat car la factorielle est des multiplications
```

```
Successives et le résultat peut être un chiffre très grand*/
```

```
do
```

```
{
```

```
printf("Donner un entier positif pour calculer sa factorielle\n");
```

```
scanf("%d", &N);
```

```
} while (N<0);
```

```
F=1;
```

```
for(i=2;i<=N;i++)
```

F=F*i ; // on boucle sur la multiplication

printf("%d!=%lld",N,F) ; /* à la sortie de la boucle on affiche le résultat avec %lld est le

format de long long int*/

}

Ecrire un algorithme qui calcule la somme d'ordre N de ***S_n*** définie comme suit en utilisant seulement les opérateurs de base (sans l'utilisation de l'opérateur de puissance).

$$S_n = \sum_{i=0}^N \frac{(-1)^{i+1}}{x^i}$$

On reprend la somme avec ses termes (i=0, i=1, i=2 jusqu'à i=N), pour mieux voir la relation entre les différentes variables qui interviennent dans la formule.

Pour N=4 on aura S4=-1+(1/x¹)+(-1/x²)+(1/x³)+(-1/x⁴)

Avec N et X sont données par l'utilisateur.

L'algorithme :

Algorithme Som_Suite;

```

Var i,N,np:entier;
    Px,X,Sn :reel ;
Debut
    Repeter
    Ecrire('Donner un entier') ;
    Lire(N) ;
    Jusqu'à N>=0 ;
    Repeter
    Ecrire('Donner un réel') ;
    Lire(X) ;
    Jusqu'à X<>0 ;

    np←-1 ; /* np représente  $(-1)^{i+1}$  , lorsque i est pair i+1 est impair alors  $(-1)^{i+1} < 0$ 
               Lorsque i est impair i+1 est pair alors  $(-1)^{i+1} > 0$  */
    Sn←-1 ; //valeur initiale de Sn pour i=0
    Px←1 ; //puissance de X pour i=0
    Pour I←1 à N Faire
        np← - np; Px←Px*X ;
        Sn←Sn+np/Px ;
    Fait ;

    Ecrire('La somme Sn= ', Sn) ;

Fin.

```

Programme :

```

#include<stdio.h>

main()

{ int N,i,np;

float Px,x,Sn ;

do

{

```

```
printf("Donner un entier positif\n") ;
scanf("%d", &N);
} while (N<0);
do
{
printf("Donner un réel\n") ;
scanf("%f", &x);
} while (X==0);
np=-1 ;
Sn=-1 ;
Px=1 ;
for(i=1;i<=N;i++)
{
np=-np;
Px=Px*x ;
Sn=Sn+np/Px ;
}
printf("La somme Sn= %0.2f", Sn) ;
}
```

LES TABLEAUX EN C

Les vecteurs en C :

1-Declaration :

Type identificateur [taille]



Nombre d'éléments du vecteur

Nom du vecteur

Type des éléments du vecteur : int, float, char

Exemple : Déclarer un vecteur de 20 entiers.

```
int T[20];
```

2-Acces aux éléments d'un vecteur :

Pour accéder à un élément d'un vecteur on spécifie le nom du vecteur et la position de l'élément dans le vecteur, la syntaxe est la suivante :

Nom _vecteur [indice de la case contenant l'élément]

Sachant que l'indice du premier élément d'un vecteur est égal à 0 en C.

Exemple : soit T un vecteur d'entiers :

10	14	25	3	7	28	11
0	1	2	3	4	5	6

Ce vecteur est de taille 7(contient 7 éléments au maximum)

T[0]=10 c'est le premier élément du vecteur .

T[1]=14 c'est le deuxième élément du vecteur.

T[2]=25 c'est le troisième élément du vecteur.

.....

T[6]=11 c'est le septième et dernier élément du vecteur.

Lecture d'un vecteur :

Pour remplir un vecteur, on doit lire plusieurs éléments donc on utilise une boucle. La boucle **for** est la plus appropriée (on peut utiliser les autres boucles).

Exemple :

```
int V[10] ;  
  
int i ;  
  
for (i=0 ;i<10 ; i++)  
  
scanf("%d",&V[i]) ;
```

Ecriture d'un vecteur :

Pour afficher un vecteur, on doit écrire plusieurs éléments donc on utilise une boucle. La boucle **for** est la plus appropriée (on peut utiliser les autres boucles).

Exemple :

```
int V[10] ;  
  
int i ;  
  
for (i=0 ;i<10 ; i++)  
  
printf("%d",V[i]) ;
```

Exercice :

Écrire un programme en C qui calcule la somme des éléments d'un vecteur de N entiers (N<=100) ;

Algorithme somme ;
Var
 T :tableau[1..100]d' entier ;
 i,N :entier ;
 S :entier ;
Debut
Repeter
 ecrire('donner la taille du tableau <=100') ;
 Lire(N); jusqu'a N>0 et N<=100 ;
Pour i←1 a N faire
 Lire (T[i]) ;
Fait ;
 S←0 ;
Pour i←1 a N faire
 S←S+T[i] ;
Fait ;
 Ecrire ('la somme =',S) ;
Fin.

Programme :

```
#include<stdio.h>
main()
{ int i,s,N ;
  int T[100] ;
  do
  {
    printf(" donner la taille du vecteur ") ;
    scanf("%d",&N) ;
  } while (N<=0 || N>100) ;
  // Remplissage du tableau
  for(i=0 ;i<N ; i++)
  {
    printf("T[%d] = ",i) ;
    scanf("%d",&T[i]) ;
  }
  // la somme des éléments du tableau
  s=0 ;
  for(i=0 ;i<N ; i++)
  s=s+T[i] ;
  printf("la somme des éléments du vecteur est %d", s) ;
}
```

C:\Users\DELL\Desktop\CoursALGO1_L1\cours_td_chapitres\chap3\tp\som_vecteur.exe

```
donner la taille du vecteur 4
T[1] = 10
T[2] = 20
T[3] = 30
T[4] = 40
la somme des éléments du vecteur est 100
-----
Process exited after 11.1 seconds with return value 40
Appuyez sur une touche pour continuer...
```

Exercice à préparer :

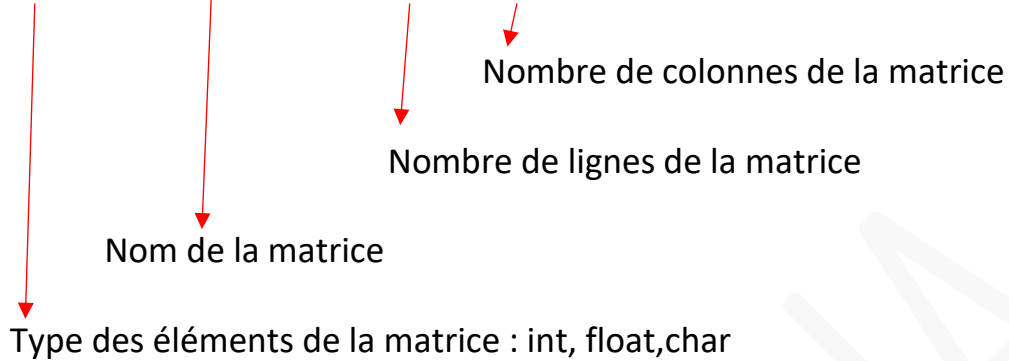
Soit un vecteur qui contient les notes de N étudiants ($N \leq 60$). Écrire un programme en C qui permet de :

- calculer la moyenne des notes du tableau.
- déterminer le nombre d'étudiants qui ont une note supérieur à cette moyenne.

LES MATRICES EN C (tableaux à deux dimensions) :

1-Declaration :

Type identificateur [taille1][taille2]



Exemple : Déclarer une matrice d'entiers, de 10 lignes de 20 colonnes.

```
int MAT[10][20];
```

2-Acces aux éléments d'une matrice :

Pour accéder à un élément d'une matrice on spécifie le nom de la matrice, le numéro de la ligne de l'élément et son numéro de colonne, la syntaxe est la suivante :

Nom _matrice [indice de ligne][indice de colonne]

Sachant que l'indice de ligne et de colonne du premier élément d'une matrice est égal à 0 en C.

Exemple : soit MAT la matrice suivante :

	0	1	2
0	10	20	14
1	9	8	7
2	4	12	13
3	6	5	74

MAT[0][0]=10 c'est le premier élément de la première ligne de la matrice.

MAT[0][2]=14 c'est le dernier élément de la première ligne de la matrice .

.....

MAT[3][0]=6 c'est le premier élément de la dernière ligne de la matrice.

MAT[3][2]=74 c'est le dernier élément de la dernière ligne de la matrice .

Lecture d'une matrice :

On peut remplir la matrice par ligne (on remplit la première ligne, puis la deuxième ligne, ainsi de suite jusqu'à la dernière ligne).

On peut aussi remplir la matrice par colonne (on remplit la première colonne, puis la deuxième colonne, ainsi de suite jusqu'à la dernière colonne).

Dans l'exemple qui va suivre on va faire un remplissage par ligne

Exemple :

```
int MAT[4][5] ;  
  
int i,j ;  
  
for (i=0 ;i<4 ; i++)  
    for (j=0 ;j<5 ; j++)  
    { printf("MAT[%d][%d]=",i,j)  
      Scanf("%d",&MAT[i][j]);  
    }
```

Ecriture d'une matrice :

On peut afficher une matrice par ligne (on affiche la première ligne, puis la deuxième ligne, ainsi de suite jusqu'à la dernière ligne).

On peut aussi afficher une matrice par colonne (on affiche la première colonne, puis la deuxième colonne, ainsi de suite jusqu'à la dernière colonne).

Dans l'exemple qui va suivre on va faire l'affichage par ligne.

Exemple :

```
int MAT[4][5] ;

int i,j ;

for (i=0 ;i<4 ; i++)

{

    for (j=0 ;j<5 ; j++)

        printf("%d \t",MAT[i][j]);

    printf("\n ") ;

}
```

Exercice

Ecrire un programme qui permet de calculer la trace d'une matrice carrée NxN avec $N \leq 30$.

```
#include<stdio.h>

main()

{ int i,j,s,N ;

  int MAT[30][30] ;

  do

  {

      printf(" donner la taille de la matrice ") ;

      scanf("%d",&N) ;
```

```

} while (N<=0 || N>30) ;

// Remplissage par ligne de la matrice
for(i=0 ;i<N ; i++)
    for (j=0 ;j<N ; j++)
    {
        printf("MAT[%d][%d]=" ,i,j);
        scanf("%d", &MAT[i][j]);
    }
s=0;
for(i=0 ;i<N ; i++)
s=s+MAT[i][i];
printf("la trace =%d",s);

// Affichage de la matrice
printf("la matrice introduite : ");
for (i=0 ;i<N ; i++)
{
    for (j=0 ;j<N ; j++)
        printf("%d \t",MAT[i][j]);
    printf("\n ") ;
}
}

```

The screenshot shows a Windows command prompt window titled "C:\document du_bureau\CoursALGO1_1\cours_td_chapitres\chap3\tp6.exe". The program prompts the user to "donner la taille de la matrice 4". It then reads 16 integers for a 4x4 matrix. The output shows the matrix elements and the trace (sum of diagonal elements) as 34. The matrix is displayed as follows:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

The program then prints "la matrice introduite :" followed by the matrix elements. At the bottom, it shows "Process exited after 21.59 seconds with return value 4" and "Appuyez sur une touche pour continuer...". The status bar at the bottom indicates "Sel: 0 Lines: 96 Length: 1705 Insertion Done parsing in 0,016 seconds".

Exercice à préparer :

Ecrire un programme qui affiche le triangle de Pascal de degré N. (N étant un entier strictement positif). Exemple pour N=4

```

1
1    1
1    2    1
1    3    3    1
1    4    6    4    1

```

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int T[20][20];
```

```
int i,N,j;
```

```
do{
```

```
printf("donner l'ordre du triangle de pascal: ");
```

```
scanf("%d", &N);
```

```
} while( N<0 || N>20 );
```

```
for(i=0;i<=N;i++)
```

```
for(j=0; j<=N;j++)
```

```
if((i==j) || (j==0)) T[i][j]=1;
```

```
else T[i][j]=T[i-1][j-1]+T[i-1][j];
```

```
/* Afficher le resultat*/
```

```
for(i=0;i<=N;i++)
```

```
{
```

```
for(j=0;j<=i;j++)
```

```
printf("%d\t", T[i][j]);
```

```
printf("\n");
```

```
}
```

```
}
```

```

C:\document_bureau\CoursALGO1\1\cours_1d_chapitres\chap3\tp\triangle_pascal.exe
donner l'ordre du triangle de pascal: 4
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
-----
Process exited after 2.571 seconds with return value 4
Appuyez sur une touche pour continuer...

```

Les chaines de caractères

En C

Une chaine de caractère en langage C, est un tableau de caractères.

1. Déclaration :

La syntaxe pour déclarer une chaine de caractère est la suivante :

```
char nom_chaine[longueur_max_de la chaine]
```

Exemple : déclarer une chaine de longueur 5 et l'initialiser à "BIEN"

```
char ch[5]= "BIEN " ;
```

Remarque : la chaine de caractère est terminée par un caractère de fin de chaine qui s'écrit par '\0'. L'exemple précédant se présente comme suit :

'B'	'I'	'E'	'N'	'\0'
-----	-----	-----	-----	------

Donc il faut toujours penser à ajouter un caractère à la taille de la chaine.

Lire une chaine de caractère :

a)-lire avec scanf : la syntaxe est la suivante,

```
scanf("%s", idf_chaine) ;
```

Exemple

```
char ch[20] ;
```

```
scanf("%s", ch);
```

Avantage : on peut lire plusieurs données à la fois (dans un seul scanf, on peut lire des chaines, entiers, réels et caractères en même temps.

```
#include<stdio.h>
```

```

main()

{ int i,j ;

char ch1[5];

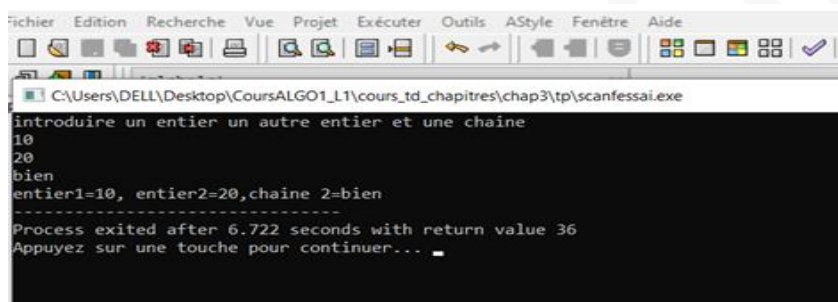
printf("introduire un entier un autre entier et une chaine\n");

scanf("%d %d %s",&i,&j,ch1);

printf("entier1=%d, entier2=%d,chaine 2=%s",i,j,ch1);

}

```



```

C:\Users\DELL\Desktop\CoursALGO1_L1\cours_td_chapitres\chap3\tp\scanfessai.exe
introduire un entier un autre entier et une chaine
10
20
bien
entier1=10, entier2=20,chaine 2=bien
-----
Process exited after 6.722 seconds with return value 36
Appuyez sur une touche pour continuer...

```

Exemple2 : lecture d'entiers et de deux chaines

```

#include<stdio.h>
#include<string.h>

main()
{ int i,j ;
char ch[5],ch1[3];

printf("introduire un entier un autre entier et deux chaines\n");

scanf("%d %d %s %s",&i,&j,ch,ch1);

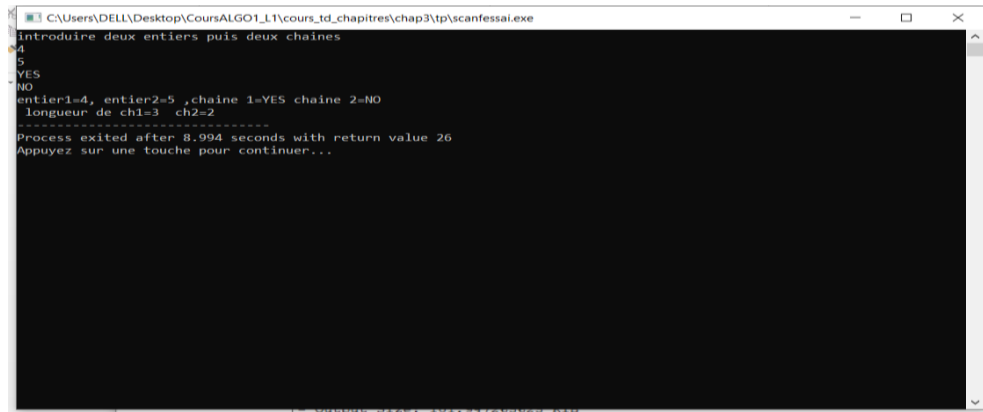
printf("entier1=%d, entier2=%d ,chaine 1=%s chaine 2=%s ",i,j,ch,ch1);

printf("\n longueur de ch1=%d ch2=%d", strlen(ch),strlen(ch1));

}

```

Remarque : lorsque vous exécutez, il faut respecter la longueur max de la chaîne (4 pour la première et 2 pour la deuxième) dans le cas contraire vous allez avoir des surprises, vous pouvez essayer et voir.



```
C:\Users\DELL\Desktop\CoursALGO1_L1\cours_td_chapitres\chap3\tp\scanfessai.exe
introduire deux entiers puis deux chaînes
4
5
YES
NO
entier1=4, entier2=5 ,chaîne 1=YES chaîne 2=NO
longueur de ch1=3 ch2=2
-----
Process exited after 8.994 seconds with return value 26
Appuyez sur une touche pour continuer...
```

Inconvénient scanf : les chaînes composées, séparées par des blancs sont tronquées.

2- lire avec gets : la syntaxe est la suivante,

gets(idf_chaine) ;

Exemple

char ch[20] ;

gets(ch) ;

Avantage : les chaînes composées, séparées par des blancs ne sont pas tronquées.

Inconvénients : on peut lire que des chaînes de caractères, et on peut faire qu'une seule lecture à la fois.

Ecrire une chaîne de caractère :

1- écrire avec printf : la syntaxe est la suivante,

printf("%s", idf_chaine) ;

Exemple

char ch[20] ;

scanf("%s", ch) ;

printf("%s",ch) ;

2- écrire avec puts : la syntaxe est la suivante,

puts(idf_chaine) ;

Exemple

```
char ch[20] ;
```

```
gets( ch) ;
```

```
puts(ch) ;
```

Inconvénient : on peut afficher une seule chaîne ou un message à la fois.

Avantage : affichage de la chaîne puis un retour à la ligne automatique.

Quelques fonctions de manipulation de chaînes de caractères :

Le prototype de ces fonctions se trouve dans le fichier **string.h** qu'il faut inclure dans l'entête du programme. `#include<string.h>`

Les fonctions que nous allons utiliser souvent :

a) strlen(s) : fournit la longueur de la chaîne **s** sans compter le '\0' final.

Exemple

```
int N ;
```

```
char nom[25] ;
```

```
gets(nom) ;
```

```
N=strlen(nom) ;
```

b) strcpy(s, t) : copie la chaîne **t** vers la chaîne **s** (t peut être une variable ou une chaîne constante)

Exemple

```
char nom1[25],nom2[20] ;
```

```
gets(nom2) ;
```

```
strcpy(nom1,nom2) ;
```

c) **strcat(s, t)** : ajoute la chaîne t à la fin de la chaîne s

Exemple :

```
#include <stdio.h>
```

```
#include <string.h>
```

```
main()
```

```
{
```

```
    char src[] = "debut";
```

```
    char dest[50] = "fin";
```

```
    strcat(dest, src);
```

```
    // Affiche le résultat de la concaténation
```

```
    printf("%s", dest);
```

```
}
```



d) **strcmp(s, t)** : compare les chaînes s et t lexico graphiquement et fournit un résultat :

 négatif : si s précède t.

 zéro : si s est égal à t.

 positif : si s suit t.

Exercice : écrire un programme qui permet de vérifier si un mot est palindrome ou non.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
main()
```

```
{char mot[10];
```

```
int i,N,j,p=1;
```

```

do

{

    printf("donner un mot non vide:\n");

    gets(mot);

} while (strlen(mot)<=0);

N=strlen(mot);i=0;

printf("le mot est de taille N=%d\n",N);

j=N-1;

while((i<j)&&(p==1))

{

    if(mot[i]!=mot[j]) p=0;

    i++; j--;

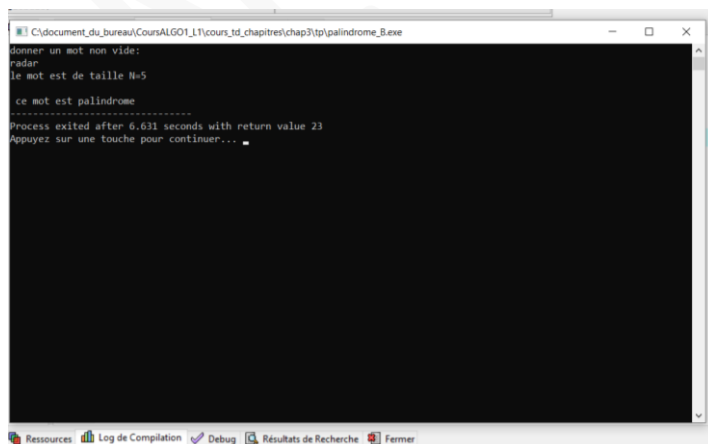
}

if (p==0) printf("\n mot non palindrome");

else printf("\n ce mot est palindrome");

}

```



```

C:\document_du_bureau\Cours\ALGO1_11\cours_td_chapitre\chap3\tp\palindrome_B.exe
donner un mot non vide:
radar
le mot est de taille N=5

ce mot est palindrome
.....
Process exited after 6.631 seconds with return value 23
Appuyez sur une touche pour continuer...

```

